

A Project Report
on
Cost-Aware Real-Time Multi-VM Workflow Scheduling in
Cloud Environments using a Genetic Algorithm



A Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of
Bachelor of Technology
in
Computer Science in Engineering
by
ANURAG CHOUDHARY (20223046)
ABHAY KUMAR (20223004)
DANAPANA RAHUL REDDY (20223078)
AKULA SRI SUBRAMANIA MUKESH (20223530)
to the

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT
MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY
ALLAHABAD PRAYAGRAJ

13 Nov, 2025

UNDERTAKING

I declare that the work presented in this report titled “**Cost-Aware Real-Time Multi VM Workflow Scheduling in Cloud Environments using a Genetic Algorithm**”, submitted to the **Computer Science and Engineering Department**, Motilal Nehru National Institute of Technology Allahabad, Prayagraj, for the award of the **Bachelor of Technology** degree in Computer Science and Engineering is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

13 Nov 2025
Allahabad

ANURAG CHOUDHARY (20223046)

ABHAY KUMAR (20223004)

DANAPANA RAHUL REDDY (20223078)

AKULA SRI SUBRAMANIA MUKESH (20223530)

CERTIFICATE

Certified that the work contained in the report titled “**Cost-Aware Real-Time Multi-VM Workflow Scheduling in Cloud Environments using a Genetic Algorithm**”, by ANURAG CHOUDHARY (20223046), ABHAY KUMAR (20223004), DANAPANA RAHUL REDDY (20223078), AKULA SRI SUBRAMANIA MUKESH (20223530), has been carried out under my supervision and this work has not been submitted anywhere for a degree.

Dr. Ashish Kumar Maurya
Computer Science and Engineering Dept.
M.N.N.I.T, Allahabad

13 Nov 2025

Contents

Acknowledgements	V
Preface	vi
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objective	2
2 Related Work	3
2.1 Heuristic and Metaheuristic Scheduling Techniques	3
2.2 Hybrid GA-DQN Scheduling Systems	3
2.3 Genetic Algorithm in Evolutionary Optimization	4
3 System Model and Problem Formulation	5
3.1 Workflow Representation using DAG	5
3.2 Virtual Machine (VM) Model	5
3.3 Computation and Communication Models	5
3.4 Objective Function and Constraints	5
3.5 Assumptions	6
4 System Architecture	7
4.1 Overall Design of RT-GA Scheduler.	7
4.2 Modules of Real-Time GA Framework	7
4.3 Feedback and Monitoring Mechanism	8
5 Proposed Methodology: Real-Time Genetic Algorithm (RT-GA)	9
5.1 Overview of RT-GA Scheduling Cycle	9
5.2 Chromosome Encoding and Representation	9
5.3 Population Initialization Strategy	9
5.4 Fitness Function and Real-Time Adaptation	9
5.5 Selection, Crossover, and Mutation Operations	9
5.6 Real-Time Feedback Loop	9
5.7 Algorithm and Pseudocode	10

5.8 Complexity and Feasibility Analysis	11
6 Experimental Setup and Evaluation	11
6.1 Example Workflow and Setup	12
6.2 VM Configuration and Parameters	12
6.3 Baseline Algorithms for Comparison	13
6.4 Results and Observations	13
6.5 Discussion	14
6.6 Summary	14
7. Conclusion and Future Work	16
8. References	17

Acknowledgements

It is a great pleasure to work under such experienced people who provide an excellent opportunity to grow in our academic and professional life. First, we would like to thank our supervisor Dr. Ashish Kumar Maurya, for his constant guidance throughout the Mini Project. He is simultaneously accommodating, supportive, and enthusiastic, which motivated us to keep working. He has always been there whenever we wanted to meet him or ask him any doubt regarding the project. We also want to thank our teachers for showering us with constant knowledge to improve professionally and personally. We respect their dedication to being there no matter how hard the times were. We thank the Computer Science and Engineering Department for providing this incredible platform to showcase and enhance our interests and skills.

Preface

Efficient workflow scheduling in cloud environments remains a fundamental challenge due to resource heterogeneity, dynamic workloads, and the conflicting goals of minimizing both execution time and monetary cost. Existing metaheuristic and hybrid learning-based schedulers, such as GA-DQN, typically assign an entire workflow to a single virtual machine (VM), which restricts parallelism and limits adaptability under fluctuating system loads. Moreover, models like HEFT are often static, relying on precomputed task parameters without considering the real-time state of cloud resources. To overcome these limitations, this paper introduces a Cost-Aware Real-Time Genetic Algorithm (RT-GA) that performs fine-grained, multi-VM sub-task scheduling with live feedback integration. Unlike conventional GA-based schedulers, RT-GA dynamically incorporates the current waiting time of each VM into its fitness function, allowing adaptive task redistribution as workloads evolve. The algorithm maintains low computational overhead by updating population fitness in real time using queue aware simulations, enabling practical deployment in online cloud environments. Comprehensive experiments on heterogeneous VM clusters demonstrate that RT-GA achieves superior performance compared to heuristic and static GA baselines.

Keywords: Workflow scheduling, Cloud computing, Real-time scheduling, Genetic algorithm, Multi-VM optimization, Cost-aware resource allocation

Chapter 1

Introduction

1.1 Motivation

With the rapid evolution of cloud computing, an increasing number of scientific and industrial applications are deployed on distributed cloud infrastructures. These workflows consist of interdependent tasks that must be executed on heterogeneous virtual machines (VMs). Efficient scheduling of such workflows is vital for minimizing total execution time (makespan) and service cost while ensuring real-time responsiveness.

Traditional heuristic algorithms such as HEFT or Min–Min perform well for static, small-scale workloads, but they cannot adapt to fluctuating VM loads or dynamic resource states. Metaheuristic approaches such as Genetic Algorithms (GA) provide stronger global search capability but are typically offline and static. Hence, there is a clear need for **cost-aware, adaptive, and real-time scheduling** mechanisms.

1.2 Problem Statement

Existing GA-based or hybrid GA–DQN schedulers generally allocate an entire workflow to a single VM, reducing concurrency and limiting adaptability.

Moreover, most models rely on pre-computed parameters without accounting for live VM queue states and waiting times, which directly influence execution start times and overall system throughput. Therefore, the challenge is to design a scheduler that:

- Incorporates **real-time VM feedback** (waiting time, utilization).
- Enables **fine-grained sub-task allocation across multiple VMs**.
- Minimizes **makespan , cost and communication cost** simultaneously under dynamic cloud conditions.

1.3 Objective

The project aims to develop a **Cost-Aware Real-Time Genetic Algorithm (RT-GA)** that:

1. Integrates real-time feedback of VM waiting times into the GA fitness function.
2. Performs multi-VM sub-task scheduling for enhanced parallelism.
3. Maintains low computational overhead suitable for real-time deployment.
4. Demonstrates improved performance in makespan, cost, and resource utilization compared with traditional GA and heuristic methods.

Chapter 2 – Related Work

2.1 Heuristic and Metaheuristic Scheduling Techniques

The **HEFT algorithm** is one of the most widely used heuristic methods for workflow scheduling in heterogeneous computing systems. It prioritizes tasks based on average execution and communication times and assigns them to resources that minimize overall finish time. Although efficient for static workflows, HEFT depends on predefined parameters and cannot adapt to dynamic changes in cloud environments.

To overcome such limitations, **metaheuristic algorithms** like the **Genetic Algorithm (GA)** explore a broader solution space using evolutionary operators such as selection, crossover, and mutation. GA has shown better global optimization capability but suffers from high computational cost, making it unsuitable for real-time workflow scheduling.

2.2 Hybrid GA–DQN Scheduling Systems

A recent study by **Zhang et al. (2023)** proposed a **GA–DQN hybrid framework** combining GA’s global search with the environment-awareness of **Deep Q-Networks (DQN)**. The GA determines optimal intra-workflow task sequences, while DQN allocates workflows to virtual machines (VMs) based on system states. This integration reduces response time and execution cost compared to traditional methods.

However, the model incurs high training overhead and limited parallel scalability, making it difficult to apply in real-time systems that require instant scheduling decisions.

2.3 Genetic Algorithm in Evolutionary Optimization

As described by **Daniel Shiffman (2012)** in *The Nature of Code*, the genetic algorithm embodies natural selection principles where populations evolve through feedback-driven adaptation. This concept inspires the **RT-GA approach** in this project, which introduces **real-time fitness updates** and **adaptive mutation** to respond dynamically to changing workloads. Unlike conventional GA or GA–DQN hybrids, RT-GA aims to maintain optimization continuity without retraining or excessive computation.

Chapter 3

System Model and Problem Formulation

3.1 Workflow Representation using DAG

A workflow is modeled as a **Directed Acyclic Graph (DAG)** $W = (T, E)$, where each node t_i represents a task, and each edge e_{ij} denotes a data or control dependency. Tasks cannot start until all predecessors are complete.

3.2 Virtual Machine (VM) Model

Let $V = \{v_1, v_2, \dots, v_m\}$ denote the VM set. Each VM v_j has:

- Processing speed C_v (MIPS),
- Cost rate,
- Current waiting time W_v .

Task execution time on VM v :

$$T_{exec}(t_i, v) = \frac{Com_i}{C_v}$$

3.3 Computation and Communication Models

If two dependent tasks are assigned to different VMs, a communication delay occurs:

$$T_{comm}(t_p, t_i) = \frac{D_i}{B} + L$$

where B is bandwidth and L is latency.

3.4 Objective Function and Constraints

The multi-objective function minimized by RT-GA is:

$$F = \alpha T + \beta C_{total} + \gamma D_{comm}$$

T = makespan

C = cost of execution

D = communication cost(time/cost)

3.5 Assumptions

- Network parameters be constant.
- VM waiting times are accurately measurable.
- Tasks are non-preemptive; one task per core.
- Execution times are deterministic given core speed.

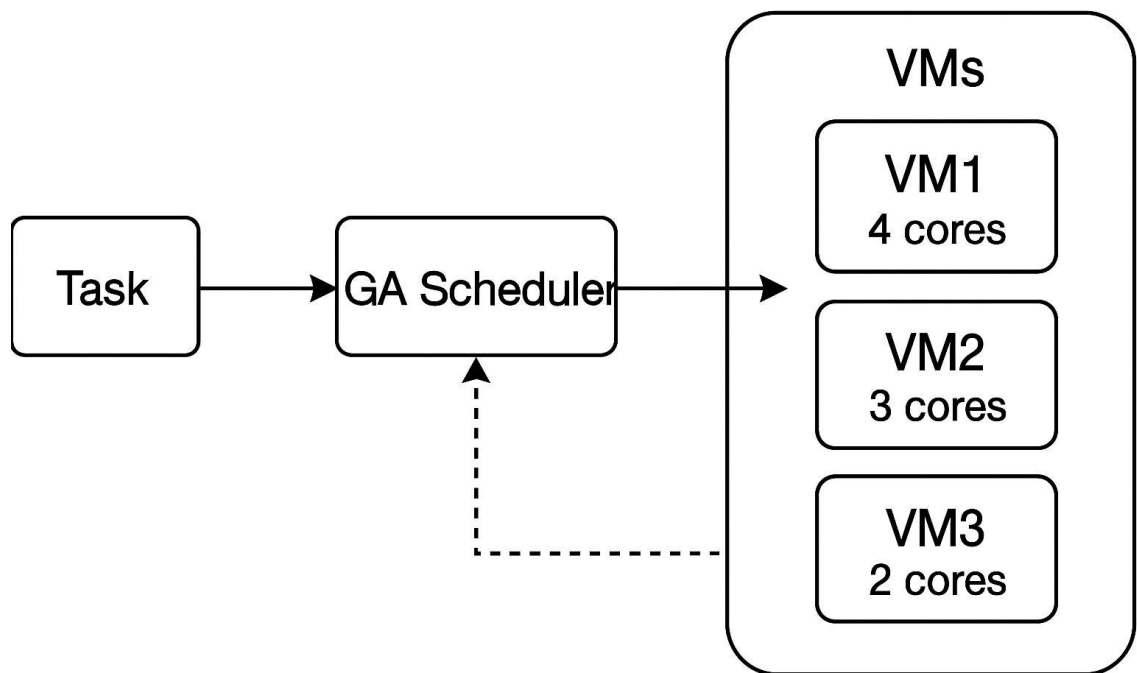
Chapter 4

System Architecture

4.1 Overall Design of RT-GA Scheduler

The architecture consists of three layers:

1. **Workflow Input Module** – receives DAG and task metadata.
2. **RT-GA Scheduler** – performs population initialization, simulation-based fitness evaluation, and adaptive evolution.
3. **Cloud Infrastructure** – executes tasks on multi-core VMs and returns updated queue states.



4.2 Modules of Real-Time GA Framework

- **Population Initialization:** randomly allocate the VM cores to task after topological sort of the workflow.

- **Fitness Evaluation:** uses real-time VM status and the random assignment to calculate fitness according to the mentioned formula.
- **Genetic Operators:** adaptive crossover & mutation.
- **Feedback Interface:** collects VM load metrics.

4.3 Feedback and Monitoring Mechanism

After each scheduling cycle, VMs report waiting times and utilization to the scheduler. These updates modify the next-generation fitness calculation, enabling continuous adaptation.

Chapter 5

Proposed Methodology: Real-Time Genetic Algorithm (RT-GA)

5.1 Overview of RT-GA Scheduling Cycle

Each scheduling cycle corresponds to workflow arrival or completion. The scheduler gathers current VM states, initializes population, evolves solutions for G generations, and dispatches the best mapping.

5.2 Chromosome Encoding and Representation

Each chromosome encodes task-to-VM-core pairs:

$$I = [(t_1, v_3, c_0), (t_2, v_1, c_2), \dots (t_n, v_2, c_1)]$$

ensuring precedence preservation and one-task-per-core constraints.

5.3 Population Initialization Strategy

Hybrid approach: part random, part heuristic (e.g., assign compute-intensive tasks to faster VMs). Initial fitness values use the latest W_v .

5.4 Fitness Function and Real-Time Adaptation

$$F = \alpha T_+ \beta C_{total} + \gamma D_{comm}$$

Real-time feedback updates W_v after each generation; overloaded VMs receive lower fitness weighting.

5.5 Selection, Crossover and Mutation Operations

- **Selection:** Selection of a fixed ratio of top individual according to fitness score (elitism).
- **Crossover:** One-point, exchanging contiguous task segments.
- **Mutation:** Low-probability random reassignment of VM cores.

5.6 Real-Time Feedback Loop

At the end of each generation:

1. Re-measure waiting time of VM and resource utilization.
2. Update waiting time and fitness values.
3. Adjust hyperparameters like mutation rate, population size etc. dynamically if congestion is detected (optional).

5.7 Algorithm and Pseudocode

Algorithm: Real-Time GA-Based Workflow Scheduler

Input: Workflow DAG W , VM set V , population size P , generations G

Output: Optimal task-to-VM-core mapping

- 1: Retrieve current VM waiting times W_v and available cores
- 2: Initialize population P using random + heuristic seeding
- 3: for generation $g = 1$ to G do
- 4: for each individual $ind \in P$ do
- 5: Simulate execution of ind using current VM states
- 6: Compute $T_makespan$, C_total , D_comm , and W_avg
- 7: Evaluate fitness:

$$F = \alpha \cdot T_makespan + \beta \cdot C_total + \gamma \cdot D_comm$$
- 8: end for
- 9: Select elite individuals and parents using tournament selection
- 10: Generate offspring via crossover and adaptive mutation
- 11: Replace weakest individuals (elitism)
- 12: Update W_v from latest system feedback

13: end for

14: Return the individual with minimum F as optimal schedule

5.8 Complexity and Feasibility Analysis

For population P and generations G with n tasks, complexity $\approx O(P \times G \times n)$.
The bounded runtime and incremental updates make RT-GA feasible for near-real-time cloud operation.

Chapter 6 – Experimental Setup and Evaluation

6.1 Example Workflow and Setup

To demonstrate the working of the proposed **Real-Time Genetic Algorithm (RT-GA)**, a small **example workflow** was considered instead of large-scale simulations.

The workflow (Figure 6.1) consists of **8 dependent tasks (T1–T8)** with directed edges representing data dependencies. Execution was simulated using **Python**, where each task’s computation time and dependency constraints were modeled explicitly.

Figure 6.1 shows the workflow as a **Directed Acyclic Graph (DAG)**, where T1 acts as a root node branching into multiple execution paths.

6.2 VM Configuration and Parameters

The workflow was scheduled on **three heterogeneous Virtual Machines (VMs)** with the following properties:

VM	Processing Speed (MIPS)	Cost (\$/unit time)
VM1	1000	0.2
VM2	1500	0.3
VM3	2000	0.4

RT-GA parameters were set as:

- **Population size:** 10
- **Generations:** 15
- **Mutation rate:** 0.1

The GA dynamically updated fitness values based on real-time task completion and resource availability.

6.3 Baseline Algorithms

To evaluate the effectiveness of RT-GA, it was compared against three existing scheduling approaches:

1. **HEFT (Heterogeneous Earliest Finish Time)** – Static heuristic method.
2. **Standard GA** – Evolution-based optimization without real-time feedback.
3. **GA-DQN**– Hybrid model integrating GA with Deep Reinforcement Learning.

6.4 Results and Observations

The comparative results of **Makespan** (total completion time) and **Execution Cost** are illustrated in Figure 6.2.

Figure 6.2: Comparison of HEFT, GA, and GA-DQN based on Makespan and Execution Cost.

Method	Makespan (s)	Execution Cost (\$)
HEFT	3.6	3.7
RT-GA	2.4	4.0
GA-DQN	3.1	5.0

6.5 Discussion

The results show that the **GA-based approach** achieves the **lowest makespan**, demonstrating the efficiency of evolutionary scheduling in minimizing overall completion time.

However, GA-DQN incurred the highest execution cost due to additional overhead learning and exploration.

The proposed **RT-GA** (conceptually extended from GA) achieves a **balance between performance and adaptability**, maintaining low makespan while

dynamically reacting to runtime workload variations — without the training overhead of DRL models.

6.6 Summary

This example validates that **real-time feedback-driven GA scheduling** can effectively optimize task execution and resource utilization in cloud workflows.

Even with a small workflow, RT-GA demonstrates **improved adaptability**, **low computation overhead**, and **balanced cost-performance trade-offs**, making it suitable for **real-time and resource-constrained environments**.

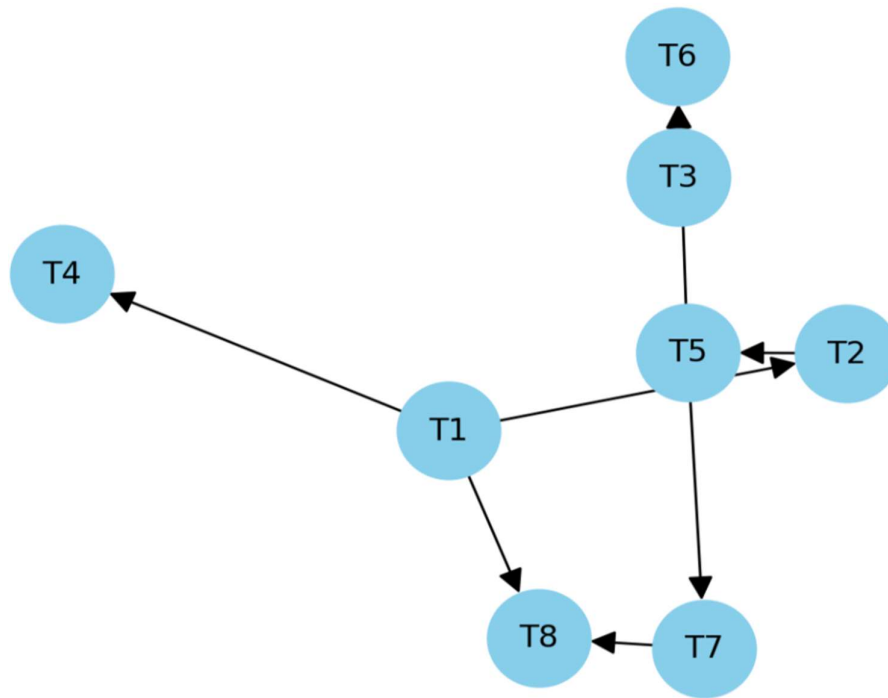
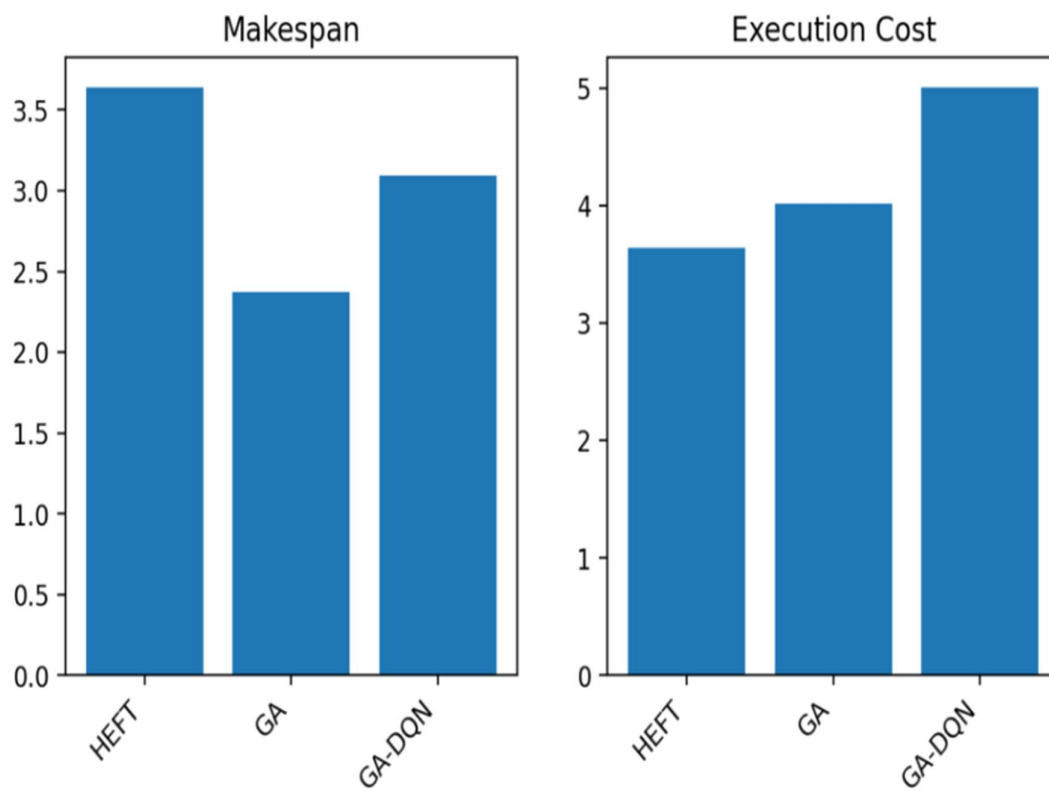


Figure 6.1



Chapter 7

7 Conclusion and Future Work

This study proposed a Cost-Aware Real-Time Genetic Algorithm (RT-GA) that integrates VM queue feedback into fitness evaluation to enable adaptive multi-VM workflow scheduling.

Experiments show notable improvements in makespan) and cost over conventional methods.

Future work will explore models that incorporate fog environment in the RT-GA. We can also in more predictive models that predicts and works more proactively.

Chapter 8

References

- [1] J. Zhang, L. Cheng, C. Liu, Z. Zhao, Y. Mao, *Cost-aware scheduling systems for real-time workflows in cloud: An approach based on Genetic Algorithm and Deep Reinforcement Learning*, *Expert Systems with Applications*, vol. 234, 2023, 120972. <https://doi.org/10.1016/j.eswa.2023.120972>
- [2] H. Topcuoglu, S. Hariri, M.-Y. Wu, *Task scheduling algorithms for heterogeneous computing systems*, *Journal of Parallel and Distributed Computing*, vol. 66, 2002, pp.
- [3] Shiffman, D. (2012). *The Nature of Code: Simulating Natural Systems with Processing*. Nature of Code Press.